

# Build Your First Library



# Today's Goals

- (1) Create a real importable Python package
- (2) Publish to TestPyPI / PyPI
- (3) Pip install your package

# What we are building

```
import pandas as pd
import simple_eda as eda

df = pd.read_csv("data.csv")

eda.summarize(df)
eda.missing(df)
eda.numeric_columns(df)
```

# What we are (not) building yet

## Building

one dependency: pandas  
plain Python functions  
clear README

## Skipping

CI/CD  
Testing  
Multi-dependencies

# The details of our journey

Design 4 functions

Create a package folder

Add pyproject.toml metadata

Install, build, check, upload, and share



**Question: What is a Python library?**

# Script vs Library

## Script

example:

```
python analyze.py
```

Less work

Less convenient

Limited scope

## Library

example:

```
import simple_eda
```

More work

More convenient

Broad scope

# Scope of our Library

Exploratory Data Analysis helps you understand a dataset  
Rows, columns, data types, missing values, basic statistics  
Our library will inspect data, not clean it automatically

# A tiny library can still be real

It does not need a web app

It does not need a fancy dashboard

It does not need dozens of dependencies

It only needs to be useful, installable, and importable

# Library bare minimums

1. Package name
2. Folder
3. `__init__.py` file
4. Functions
5. Metadata

# Package name & folder

You choose - name needs to be unique

# \_\_init\_\_.py

Exists in folder

Marks the folder as  
a Python package

Lets users write:

```
import simple_eda
```

# The functions

```
summarize(df)  
missing(df)  
numeric_columns(df)  
categorical_columns(df)
```

# The beginner rule

Make the first version boring

Use names people can guess

Return familiar pandas/Python objects

Improve only after the first version works

**Version 0.1.0 is allowed to be tiny.**

# What counts as success?

- A user can pip install it
- A user can import it
- A user can call one useful function
- A user can read the README and understand the example

# Module 1 checkpoint

A script is run

A library is imported

A package is a folder with importable code

pip needs package metadata

**Next: design the smallest API.**



## Module 2: Design the smallest EDA API

The API is what users remember.

# Start with the user sentence

If the first example feels simple, the library feels simple.

```
import simple_eda as eda  
eda.summarize(df)
```

# Pick boring function names

## Good names

summarize

missing

numeric\_columns

categorical\_columns

## Avoid clever names

hyper\_profile\_engine

magic\_diagnose

auto\_ai\_report

mega\_summary

# One input type at first

Support pandas DataFrames first

Do not support Polars, Spark, Dask, or Arrow yet

One clear input type reduces bugs and teaching complexity

**Beginner-friendly means fewer choices.**

# Return plain objects first

Dictionaries, lists, Series, and DataFrames are enough.

```
{  
  "rows": 100,  
  "columns": 5,  
  "column_names": ["city", "sales"]  
}
```

# Do not mutate user data

## Good

```
df.isna().sum()  
df.describe()  
list(df.columns)
```

## Bad

```
df.dropna(inplace=True)  
df["x"] = ...  
silent type conversion
```

# Keep dependencies small

Use pandas only for version 0.1.0

Every dependency adds install time and risk

Add visualization libraries only when the core library already works

**Small dependency lists are beginner-friendly and safer.**

# API surface area

Public: functions users call directly

Internal: helper functions you may change later

Expose only what you want to support

```
# public
eda.summarize(df)

# internal
_simple_helper(df)
```

# Good defaults

A function should work without many arguments

Arguments should be optional when possible

Use sensible behavior before adding configuration

# Naming checklist

- Can a beginner guess what it does?
- Is the name short?
- Is it a verb or clear noun phrase?
- Does it avoid abbreviations?
- Does it match the README example?

# Version 0.1.0 mindset

First working version

Not feature-complete

Not perfect

Installable and understandable

**The goal is momentum, not completeness.**

# Design decision record

Library name: simple\_eda

Install name: simple-eda-yourname

Core dependency: pandas

Public functions: four EDA helpers

| Name | Functions | Returns | Dependency | Install |
|------|-----------|---------|------------|---------|
|------|-----------|---------|------------|---------|

# Module 2 checkpoint

The API is tiny

The names are obvious

The input is pandas only

The library never changes the user's dataframe

**Next: create the package folder.**



## Module 3: Create the package folder

Turn loose code into an importable package.

# Final folder shape

This is the modern src layout.

```
simple-eda-project/  
├── src/  
│   ├── simple_eda/  
│   │   ├── __init__.py  
│   │   └── core.py  
├── README.md  
├── pyproject.toml  
└── LICENSE
```

# Project name vs import name

## Install name

simple-eda-yourname  
hyphens are normal  
used by pip and PyPI

## Import name

simple\_eda  
underscores are normal  
used in Python code

# Create the project root

The project root will contain pyproject.toml.

```
mkdir simple-eda-project  
cd simple-eda-project
```

# Create the package folder

The code lives inside `src/simple_eda`.

```
mkdir -p src/simple_eda  
touch src/simple_eda/__init__.py  
touch src/simple_eda/core.py
```

# Create project files

These files explain, package, and license the project.

```
touch README.md  
touch pyproject.toml  
touch LICENSE
```

# What `__init__.py` does

Marks the folder as a  
Python package

Controls the top-level  
import experience

Lets users write import  
`simple_eda as eda`

```
#  
src/simple_eda/__init__  
.py  
from .core import  
summarize
```

# What core.py does

Holds the first reusable functions

Keeps the project easy to navigate

Gives beginners one obvious file to edit

```
#  
src/simple_eda/core.py  
def summarize(df):  
    ...
```

# Why use src layout?

Prevents accidental imports from the project root

Matches many modern Python projects

Makes packaging behavior clearer later

**It looks slightly more complex, but saves confusion.**

# Check the tree

```
find .  
  
./src/simple_eda/__init__.py  
./src/simple_eda/core.py  
./README.md  
./pyproject.toml  
./LICENSE
```

# Common beginner mistake

Putting pyproject.toml inside src/simple\_eda

Forgetting \_\_init\_\_.py

Naming the folder simple-eda instead of simple\_eda

Running pip install from the wrong directory

# Naming with your own suffix

Your PyPI name must be globally unique.

```
# Package name on PyPI
simple-eda-victor
simple-eda-ana
simple-eda-yourgithub

# Import name in Python
simple_eda
```

# Module 3 checkpoint

- Project root exists
- src/simple\_eda exists
- \_\_init\_\_.py exists
- core.py exists
- README.md exists
- pyproject.toml exists
- LICENSE exists



## Module 4: Write the first EDA functions

Start with plain pandas helpers.

# Write summarize()

```
def summarize(df):  
    """Return a small summary of a pandas  
    DataFrame."""  
    return {  
        "rows": len(df),  
        "columns": len(df.columns),  
        "column_names": list(df.columns),  
        "dtypes":  
        df.dtypes.astype(str).to_dict(),  
    }
```

# What summarize() returns

Number of rows

Number of columns

Column names

Data types converted to strings

**This is simple, but immediately useful.**

# Write missing()

```
def missing(df):  
    """Return missing-value counts for each  
    column."""  
    return  
df.isna().sum().sort_values(ascending=False  
)
```

# Why missing values first?

Missing values are one of the first EDA questions

Pandas already has the building blocks

Sorting makes the output easier to scan

# Write numeric\_columns()

```
def numeric_columns(df):  
    """Return the names of numeric  
    columns."""  
    return  
    list(df.select_dtypes(include="number").col  
        umns)
```

# Write categorical\_columns()

```
def categorical_columns(df):  
    """Return object/category column  
    names."""  
    return list(  
        df.select_dtypes(include=["object",  
        "category"]).columns  
    )
```

# Optional stats()

This is optional because pandas already does it well.

```
def stats(df):  
    """Return basic descriptive statistics."""  
    return df.describe()
```

# Complete core.py

Keep the first version short.

```
def summarize(df):
    return {
        "rows": len(df),
        "columns": len(df.columns),
        "column_names": list(df.columns),
        "dtypes": df.dtypes.astype(str).to_dict(),
    }

def missing(df):
    return df.isna().sum().sort_values(ascending=False)
```

# Complete core.py continued

```
def numeric_columns(df):  
    return  
    list(df.select_dtypes(include="number").columns)  
  
def categorical_columns(df):  
    return list(  
        df.select_dtypes(include=["object",  
        "category"]).columns
```

# Expose functions in `__init__.py`

This creates the clean top-level API.

```
from .core import (  
    summarize,  
    missing,  
    numeric_columns,  
    categorical_columns,  
    stats,  
)
```

# Add `__all__`

`__all__` is a clear list of public functions.

```
__all__ = [  
    "summarize",  
    "missing",  
    "numeric_columns",  
    "categorical_columns",  
    "stats",  
]
```

# Try a tiny dataframe

```
import pandas as pd
import simple_eda as eda

df = pd.DataFrame({
    "name": ["Ana", "Bo", None],
    "age": [25, None, 31],
})

print(eda.summarize(df))
```

# Try missing values

```
print(eda.missing(df))  
  
# expected idea:  
# name      1  
# age       1  
# dtype: int64
```

# Try column helpers

```
print(eda.numeric_columns(df))  
# ["age"]  
  
print(eda.categorical_columns(df))  
# ["name"]
```

# Docstrings matter

They show up in `help()`

They help autocomplete tools

They explain purpose without a docs site

Short docstrings are enough for version 0.1.0

# No custom classes yet

Custom classes are useful later

Beginners understand dicts, lists, and pandas Series

Plain objects are easier to test and print

**Avoid architecture until you feel pain.**

# Common function mistakes

Changing df in place

Returning printed text instead of data

Adding too many optional arguments

Trying to solve every EDA problem in one function

# Module 4 checkpoint

- `core.py` has useful functions
- `__init__.py` imports them
- Users can call `eda.summarize(df)`
- Functions return simple objects
- The dataframe is not modified



## Module 5: Add pyproject.toml

Teach pip what your project is.

# What pyproject.toml does

Defines the package name and version

Lists dependencies

Tells build tools how to build

Replaces many old setup.py workflows

**pip needs metadata, not just code.**

# The build-system section

This tells Python which build backend to use.

```
[build-system]
requires = ["setuptools>=61"]
build-backend = "setuptools.build_meta"
```

# The project section

```
[project]
name = "simple-eda-yourname"
version = "0.1.0"
description = "A tiny beginner-friendly EDA
helper for pandas"
readme = "README.md"
requires-python = ">=3.9"
```

# Dependencies

pandas will be installed automatically if needed.

```
dependencies = [  
    "pandas>=1.5"  
]
```

# Setuptools src layout

This connects the src folder to the package builder.

```
[tool.setuptools]
package-dir = {"" = "src"}

[tool.setuptools.packages.find]
where = ["src"]
```

# Full minimal pyproject.toml

```
[build-system]
requires = ["setuptools>=61"]
build-backend = "setuptools.build_meta"

[project]
name = "simple-eda-yourname"
version = "0.1.0"
description = "A tiny beginner-friendly EDA helper for pandas"
readme = "README.md"
requires-python = ">=3.9"
dependencies = ["pandas>=1.5"]
```

# Choose a unique package name

PyPI names are shared globally

simple-eda may already be taken

Use your name, handle, or project suffix

Example: simple-eda-victor

**Hyphen for install. Underscore for import.**

# Version numbers

This is enough semantic versioning for beginners.

0.1.0 = first working version

0.1.1 = small fix

0.2.0 = new feature

1.0.0 = stable public version

# README: why it matters

It is the first page users read

It shows the install command

It shows the smallest working example

It makes your package feel real

# README install section

```
# simple-eda-yourname
```

```
A tiny beginner-friendly EDA helper for  
pandas.
```

```
## Install
```

```
pip install simple-eda-yourname
```

# README usage section

```
## Use

import pandas as pd
import simple_eda as eda

df = pd.read_csv("data.csv")

print(eda.summarize(df))
print(eda.missing(df))
```

# License basics

A license tells others what they can do with your code

MIT is common for small beginner projects

Use an official template if publishing publicly

Do not invent your own license text

# Metadata buys you distribution

pip can install locally

build can create a wheel

twine can upload to TestPyPI

PyPI can show your README

Other people can install it

# Module 5 checkpoint

- pyproject.toml has build-system
- project name is unique
- version is 0.1.0
- pandas dependency is listed
- README has install and usage
- LICENSE exists



## Module 6: Install locally with pip

Make your package work on your own machine first.

## Install in editable mode

Run this from the folder that contains `pyproject.toml`.

```
python -m pip install -e .
```

# What -e means

Editable install

Your package is linked to your source folder

Code changes show up without reinstalling

Great for development and workshops

# Import smoke test

The first test is simply: can Python import it?

```
python
```

```
>>> import simple_eda
```

```
>>> print(simple_eda.__all__)
```

# Test with a DataFrame

```
import pandas as pd
import simple_eda as eda

df = pd.DataFrame({
    "city": ["SF", "LA", None],
    "sales": [100, 200, None],
})

eda.summarize(df)
eda.missing(df)
```

# Common error: package not found

You may not have installed it yet

You may be in the wrong folder

You may be using the wrong Python environment

```
ModuleNotFoundError: No  
module named  
'simple_eda'
```

# Fix: install from the project root

The project root contains pyproject.toml.

```
cd simple-eda-project  
python -m pip install -e .
```

# Common error: wrong environment

Always prefer `python -m pip` to match pip with the active Python.

```
which python  
which pip  
python -m pip --version
```

# Uninstall and reinstall

Useful when testing package metadata changes.

```
python -m pip uninstall simple-eda-yourname  
python -m pip install -e .
```

# Use it in notebooks

After local install,  
notebooks can import it  
This is already useful  
before publishing  
Local packages can  
reduce notebook  
copy/paste

```
import simple_eda as  
eda  
  
eda.summarize(df)
```

# Add a tiny test file

```
# test_example.py
import pandas as pd
import simple_eda as eda

df = pd.DataFrame({"age": [10, 20]})
print(eda.numeric_columns(df))
```

# Local install is a milestone

Your code is no longer just a script

pip knows about your project

Python can import it from anywhere in the environment

**You built a local pip-installable library.**

# Module 6 checkpoint

- `python -m pip install -e .` works
- `import simple_eda` works
- `test_example.py` runs
- notebook import works
- you know how to debug environment issues



## Module 7: Build a real package file

Create the artifacts that package indexes store.

# Install the build tool

The build package creates distributable files.

```
python -m pip install build
```

# Build the package

Run from the project root.

```
python -m build
```

# The dist folder

These are the files you upload later.

```
dist/  
├── simple_eda_yourname-0.1.0.tar.gz  
└──  
simple_eda_yourname-0.1.0-py3-none-any.whl
```

# Wheel vs source distribution

## Wheel

.whl file

fast install

most users get this

## Source distribution

.tar.gz file

source archive

used when wheel is  
unavailable

# Test the wheel locally

Never upload before testing the built file.

```
python -m pip install dist/*.whl
```

```
python -c "import simple_eda; print(simple_eda.__all__)"
```

# Clean rebuild

Use this when build files seem stale.

```
rm -rf build dist *.egg-info src/*.egg-info  
python -m build
```

# Inspect what you built

This confirms your installed package metadata.

```
python -m pip show simple-eda-yourname  
# check name, version, location, dependencies
```

# Do not commit dist files at first

dist/ is generated output

build/ is generated output

\*.egg-info is generated output

Add them to .gitignore for normal development

# Version bump before rebuild

Package indexes do not let you overwrite the same version.

```
# pyproject.toml
version = "0.1.1"

rm -rf build dist *.egg-info src/*.egg-info
python -m build
```

# Build failure checklist

- pyproject.toml is valid TOML
- package-dir points to src
- src/simple\_eda exists
- README.md exists if referenced
- dependencies list has valid strings

# Module 7 checkpoint

- `python -m build` works
- `dist/` contains `.whl` and `.tar.gz`
- the wheel installs locally
- `import` still works after wheel install



## Module 8: Publish safely

Practice on TestPyPI before uploading to real PyPI.

Build tiny. Install locally. Publish safely.

# Why TestPyPI first?

It is a practice package index

You can test upload commands

You can test install commands

Mistakes are less painful than on real PyPI

**Publish to TestPyPI before PyPI.**

# Create accounts

- Create a TestPyPI account
- Create a PyPI account
- Enable two-factor authentication
- Create an API token
- Store the token safely

# Install Twine

Twine uploads your dist files.

```
python -m pip install twine
```

# Check the package

Fix errors before uploading.

```
python -m twine check dist/*
```

# Upload to TestPyPI

Use your TestPyPI API token when prompted.

```
python -m twine upload --repository  
testpypi dist/*
```

# Install from TestPyPI

```
python -m pip install --index-url  
https://test.pypi.org/simple/  
--extra-index-url https://pypi.org/simple/  
simple-eda-yourname
```

# Why extra-index-url?

TestPyPI may not have pandas  
extra-index-url lets pip find dependencies on real PyPI  
Your package still comes from TestPyPI

# Test the TestPyPI install

If this works, your package is truly index-installable.

```
python -c "import simple_eda as eda;  
print(eda.__all__)"
```

# Upload to real PyPI

Only do this after TestPyPI works.

```
python -m twine upload dist/*
```

# Install from real PyPI

This is the public finish line.

```
pip install simple-eda-yourname  
python -c "import simple_eda"
```

# Publishing safety

- Use API tokens, not passwords
- Do not upload secrets
- Do not commit tokens
- Review dependencies
- Protect your maintainer account

# **pip install can be risky**

Packages can execute code during build or runtime

Typosquatting can trick users

Compromised maintainer accounts can publish bad updates

Fewer dependencies means less attack surface

**Small packages are easier to trust.**

# Module 8 checkpoint

- twine check passes
- TestPyPI upload works
- TestPyPI install works
- real PyPI upload is optional until ready
- account security is enabled



## Module 9: Use, improve, and share

Make the tiny project useful without making it bloated.

# Add an example notebook

Examples teach faster than long prose.

```
examples/basic_usage.ipynb
```

```
import pandas as pd  
import simple_eda as eda
```

```
df = pd.read_csv("data.csv")  
eda.summarize(df)  
eda.missing(df)
```

# Add basic tests later

Testing can come after the first package works.

```
def test_numeric_columns():  
    df = pd.DataFrame({  
        "name": ["A", "B"],  
        "age": [10, 20],  
    })  
    assert eda.numeric_columns(df) == ["age"]
```

# Good next features

Add features that stay simple.

```
eda.quick_report(df)
eda.top_categories(df, column="city")
eda.column_summary(df, column="age")
eda.duplicate_rows(df)
```

# Features to avoid at first

Full HTML reports

Plugin systems

Every dataframe backend

Complex configuration

AI-generated analysis

Dozens of dependencies

# Final project checklist

- `src/simple_eda/__init__.py` exists
- `src/simple_eda/core.py` exists
- `pyproject.toml` exists
- README has install and usage
- `pip install -e .` works
- `python -m build` works
- `twine check dist/*` works

# Capstone assignment

Build version 0.1.0 of your own tiny EDA package

Include at least three EDA functions

Make it importable and pip-installable

Publish to TestPyPI or PyPI

Share the install command and one example